

SANYAM ARORA

Capstone Assignment-1 : Report

Step-1 : I have created a storage account named **'sanyamdata'**, with **Azure Data Lake Storage Gen2** enabled as the primary service, and **hierarchical namespace** support activated.

[Home](#) > [Storage accounts](#) >

Create a storage account ...

manage your storage account together with other resources.

Subscription *	LabsKraft
Resource group *	LabsKraft2027
Create new	
Instance details	
Storage account name * ⓘ	sanyamdata
Region * ⓘ	(Asia Pacific) South India
Deploy to an Azure Extended Zone	
Primary service ⓘ	Azure Blob Storage or Azure Data Lake Storage Gen 2
Performance * ⓘ	☒ Standard: Recommended for most scenarios (general-purpose v2 account)
	☐ Premium: Recommended for scenarios that require low latency.
Redundancy * ⓘ	Locally-redundant storage (LRS)

Step-2: I have added a container named **'raw'** in the storage account, and within it, I created a directory called **'bronzelayer'**. Inside **bronzelayer**, I have ingested a CSV file named **retail_iot_data.csv**.

Reason: I placed the retail_iot_data.csv file inside the bronzelayer directory within the raw container to ensure that Auto Loader's schema and checkpoint metadata are scoped locally. This avoids conflicts in the root container and aligns with Unity Catalog's best practices for managing schema evolution and ingestion paths.

[Home](#) > [sanyamdata](#) | Containers >

sanyambronzelayer

Container

Overview

Diagnose and solve problems

Access Control (IAM)

Settings

+ Add Directory

↑ Upload

↻ Refresh

🗑 Delete

📄 Copy

📄 Paste

🔄 Rename

🔑 Acquire lease

🔑 Break lease

🔗 Edit columns

sanyambronzelayer

Authentication method: Access key ([Switch to Microsoft Entra user account](#))

Only show active objects

Showing all 1 items

☐	Name	Last modified	Access tier	Blob type	Size	Lease state	
☐	retail_iot_data.csv	8/25/2025, 10:05:06 AM	Hot (Inferred)	Block blob	130.76 KiB	Available	...

Step 3: I configured **IAM permissions** on my Azure Storage Account (sanyamdata) by creating a **Databricks connector** and assigning the **Storage Blob Data Contributor** role to the **managed identity** used by Databricks.

Purpose: I assigned the Storage Blob Data Contributor role to Databricks' managed identity to ensure secure and authorized access to my data lake. This is a critical step for enabling Delta Live Tables to read, write, and manage data across the Bronze, Silver, and Gold layers in the medallion architecture, while maintaining compliance with Unity Catalog's access and governance requirements.

[Home](#) > [sanyamdata | Access Control \(IAM\)](#) >

Add role assignment

[Role](#)
[Members](#)
[Conditions](#)
[Review + assign](#)

A role definition is a collection of permissions. You can use the built-in roles or you can create your own custom roles. [Learn more](#)

[Job function roles](#)
[Privileged administrator roles](#)

Grant access to Azure resources based on job function, such as the ability to create virtual machines.

×
Type: All
Category: All

Name ↑↓	Description ↑↓	Type ↑↓	Category ↑↓	Details
Storage Blob Data Contributor	Allows for read, write and delete access to Azure Storage blob containers and data	BuiltInRole	Storage	View
Storage Blob Data Owner	Allows for full access to Azure Storage blob containers and data, including assigning POSIX a...	BuiltInRole	Storage	View
Storage Blob Data Reader	Allows for read access to Azure Storage blob containers and data	BuiltInRole	Storage	View

Showing 1 - 3 of 3 results.

[Home](#) > [sanyamdata | Access Control \(IAM\)](#) >

Add role assignment

[Role](#)
[Members](#)
[Conditions](#)
[Review + assign](#)

Selected role

Storage Blob Data Contributor

Assign access to

☐ User, group, or service principal
 ☒ Managed identity

Members

[+ Select members](#)

Name	Object ID	Type
No members selected		

Description

Optional

Showing a filtered list of roles because your permissions include a condition. [Learn more](#)

[View my access](#)



sanyam-connector

Access Connector for Azure Databricks



Refresh



Delete

Overview

Activity log

Access control (IAM)

Tags

Resource visualizer

Settings

Automation

Help

Essentials

Resource group [\(move\)](#)

[LabsKraft2027](#)

Location

West US 2

Subscription [\(move\)](#)

[LabsKraft](#)

Subscription ID

d008c7cc-9795-4292-bf79-74ae52cda63d

Tags [\(edit\)](#)

[Add tags](#)

State

Succeeded

Resource ID

/subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resourceGroups/...

[JSON View](#)

Copied

Add role assignment

Role

Members

Conditions

[Review + assign](#)



Showing a filtered list of roles because your permissions include a condition. [Learn more](#)
[View my access](#)

Selected role

Storage Blob Data Contributor

Assign access to



User, group, or service principal



Managed identity

Members

[+ Select members](#)

Name

Object ID

No members selected

Description

Optional

[Review + assign](#)

[Previous](#)

[Next](#)

Select managed identities



Some results might be hidden due to your ABAC condition.

Subscription *

[LabsKraft](#)

Managed identity

All system-assigned managed identities (6)

[sanyam](#)

Selected members:



sanyam-connector
/subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resourceGroups/... [Remove](#)

[Select](#)

[Close](#)

[Feedback](#)

Step-4: I created a **Unity Catalog** named “Sanyam_capstone1_catalog” to manage data governance across my Databricks workspace. Before creating the catalog, I set up an **external location** named “Sanyam_capstone1_ext_loc”, which points to my Azure Data Lake Storage Gen2 (sanyamdata).

Purpose: I created an external location (xyz) before setting up the Unity Catalog (abc) to securely link my Azure Data Lake Storage Gen2 with Databricks. This step is essential for enabling Unity Catalog to manage access, enforce governance, and organize data across the Bronze, Silver, and Gold layers of the medallion architecture.

The external location acts as the **base path** for organizing your Bronze, Silver, and Gold layers. It enables:

- Structured data management.
- Schema enforcement and lineage tracking.
- Integration with Auto Loader and Delta Live Tables.

[External Locations](#) >

Create a new external location

An external location allows you to access your data stored in cloud storage (e.g. Azure Data Lake Storage). You will need the cloud storage path and a paired credential (e.g. managed identity) which gives access to that path [Learn more](#)

External location name*

sanyam_capstone1_ext_loc

Storage type*

Azure Data Lake Storage

URL* ⓘ

Enter the bucket path that you want to use as the external location. Note: This must be an ADLS Gen2 storage account with a hierarchical namespace

abfss://sanyamraw@sanyamdata.dfs.core.windows.net/

Copy from DBFS

Storage credential* [Learn more](#)

Provide a storage credential capable of accessing the URL

sanyam_capstone1_ext_loc_azuremanagedidentity_1756098373931 (Managed Identity)

Connector Id: /subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resourceGroups/LabsKraft2027/providers/Micros...

User Assigned Managed Identity Id:

Step-5: I have created a new Notebook in my workspace and have wrote codes for all three layers:

Bronze layer code:

```
from dlt import table

import pyspark.sql.functions as F

from pyspark.sql.types import StructType, StructField, StringType, DoubleType, IntegerType, TimestampType

iot_schema = StructType([

    StructField("OrderID", StringType(), True),

    StructField("CustomerID", StringType(), True),

    StructField("Region", StringType(), True),

    StructField("StoreID", StringType(), True),

    StructField("Product", StringType(), True),

    StructField("Quantity", IntegerType(), True),

    StructField("UnitPrice", DoubleType(), True),

    StructField("SalesAmount", DoubleType(), True),

    StructField("OrderTime", StringType(), True),    # can cast to Timestamp later in Silver

    StructField("DeviceType", StringType(), True),

    StructField("Temperature", DoubleType(), True),

    StructField("DeviceStatus", StringType(), True),

    StructField("AnomalyFlag", IntegerType(), True),

    StructField("AnomalyType", StringType(), True)

])

raw_path = "abfss://sanyamraw@sanyamdata.dfs.core.windows.net/bronzelayer/"

schema_path = "abfss://sanyamraw@sanyamdata.dfs.core.windows.net/dlt_schemas/bronze_orders_sanyam"

checkpoint_path = "abfss://sanyamraw@sanyamdata.dfs.core.windows.net/dlt_checkpoints/bronze_orders_sanyam"

@table(

    name="sanyam_capstone1_catalog.capstone_retail_sanyam.bronze_layer",

    comment="Bronze table - raw ingested IoT retail data via Auto Loader"

)

def bronze_orders():

    return (

        spark.readStream
```

```

        .format("cloudFiles")

        .option("cloudFiles.format", "csv")

        .option("header", True)

        .option("cloudFiles.schemaLocation", schema_path)

        .option("checkpointLocation", checkpoint_path)

        .schema(iot_schema)

        .load(raw_path)

    )

```

Summary of code: This Bronze layer ingests raw IoT retail data from CSV files stored in Azure Data Lake Storage Gen2 using Auto Loader. It defines a structured schema to parse fields like sales, device status, and anomalies. The data is read as a streaming source, enabling real-time ingestion. Schema and checkpoint metadata are managed separately to support fault tolerance and schema evolution. The ingested data is registered as a managed Delta table in Unity Catalog for downstream processing.

Silver layer code:

```

@dlt.table(

    name = "capstone_retail_sanyam.silver_layer",

    comment="Silver table - cleaned & typed data"

)

def silver_layer():

    df = dlt.read("sanyam_capstone1_catalog.capstone_retail_sanyam.bronze_layer")

    return (

        df.withColumn("Quantity", F.col("Quantity").cast("int"))

        .withColumn("UnitPrice", F.col("UnitPrice").cast("double"))

        .withColumn("OrderTime", F.to_timestamp("OrderTime")) # likely what you want

    )

```

Summary of Code: The Silver layer transforms raw data from the Bronze table by applying type casting and timestamp conversion. It ensures that fields like Quantity and UnitPrice are correctly typed as integers and doubles, respectively. The OrderTime field is converted from string to timestamp for accurate time-based analysis. This layer prepares clean, structured data for downstream aggregation and reporting in the Gold layer.

Gold layer code:

```
from dlt import table, read
```

```
import pyspark.sql.functions as F
```

1. Daily / Monthly Sales per Region

```
@table(
    name="sanyam_capstone1_catalog.capstone_retail_sanyam.gold_sales_region",
    comment="Daily & Monthly Sales per Region"
)

def gold_sales_region():
    df = dlt.read("sanyam_capstone1_catalog.capstone_retail_sanyam.silver_layer")

    return (
        df.withColumn("OrderDate", F.to_date("OrderTime"))
        .withColumn("YearMonth", F.date_format("OrderDate", "yyyy-MM"))
        .groupBy("Region", "OrderDate", "YearMonth")
        .agg(F.sum("SalesAmount").alias("TotalSales"))
    )
```

2. Device Anomaly Trend per Store

```
@table(
    name="sanyam_capstone1_catalog.capstone_retail_sanyam.gold_anomaly_trends",
    comment="Device Anomaly Trend per Store"
)

def gold_anomaly_trends():
    df = dlt.read("sanyam_capstone1_catalog.capstone_retail_sanyam.silver_layer")

    return (
        df.withColumn("OrderDate", F.to_date("OrderTime"))
        .groupBy("StoreID", "OrderDate", "DeviceType", "AnomalyType")
        .agg(F.count("*").alias("AnomalyCount"))
    )
```

3. Conversion Rate Impact when devices fail

```
@table(
    name="sanyam_capstone1_catalog.capstone_retail_sanyam.gold_conversion_impact",
    comment="Conversion rate impact of device failures"
)

def gold_conversion_impact():
    df = dlt.read("sanyam_capstone1_catalog.capstone_retail_sanyam.silver_layer")

    total_orders = df.groupBy("StoreID").agg(F.count("*").alias("TotalOrders"))

    failed_orders = (
        df.filter(df.AnomalyFlag == 1)
        .groupBy("StoreID")
        .agg(F.count("*").alias("FailedOrders"))
    )

    return (
        total_orders.join(failed_orders, "StoreID", "left")
        .withColumn("FailedOrders", F.coalesce(F.col("FailedOrders"), F.lit(0)))
        .withColumn("FailureRate", F.col("FailedOrders") / F.col("TotalOrders"))
    )
```

4. Store Tier Classification

```
@table(
    name="sanyam_capstone1_catalog.capstone_retail_sanyam.gold_store_tiers",
    comment="Store tiers based on total sales (High / Medium / Low performers)"
)

def gold_store_tiers():
    df = (
        dlt.read("sanyam_capstone1_catalog.capstone_retail_sanyam.silver_layer")
        .groupBy("StoreID")
        .agg(F.sum("SalesAmount").alias("TotalSales"))
    )

    return (
        df.withColumn(
            "StoreTier",
            F.when(F.col("TotalSales") > 100000, "High")
```



```

        .when(F.col("TotalSales") > 50000, "Medium")

        .otherwise("Low")

    )

)

```

5. Weighted Average Sales vs Anomalies

```

@table(

    name="sanyam_capstone1_catalog.capstone_retail_sanyam.gold_weighted_sales_anomalies",

    comment="Weighted average of sales vs anomalies per store"

)

def gold_weighted_sales_anomalies():

    df = dlt.read("sanyam_capstone1_catalog.capstone_retail_sanyam.silver_layer")

    return (

        df.groupBy("StoreID")

        .agg(

            F.sum("SalesAmount").alias("TotalSales"),

            F.sum(F.when(F.col("AnomalyFlag") == 1, 1).otherwise(0)).alias("TotalAnomalies")

        )

        .withColumn(

            "SalesPerAnomaly",

            F.when(F.col("TotalAnomalies") > 0, F.col("TotalSales") / F.col("TotalAnomalies"))

            .otherwise(F.lit(None))

        )

    )

```

Summary of Code: The Gold layer performs advanced aggregations and analytics on cleaned data from the Silver layer to support business intelligence and reporting:

1. **Sales Aggregation**
Calculates daily and monthly total sales per region for trend analysis.
2. **Anomaly Tracking**
Tracks device anomalies per store by date, device type, and anomaly type.
3. **Conversion Impact**
Measures the impact of device failures on conversion rates by comparing failed vs total orders per store.

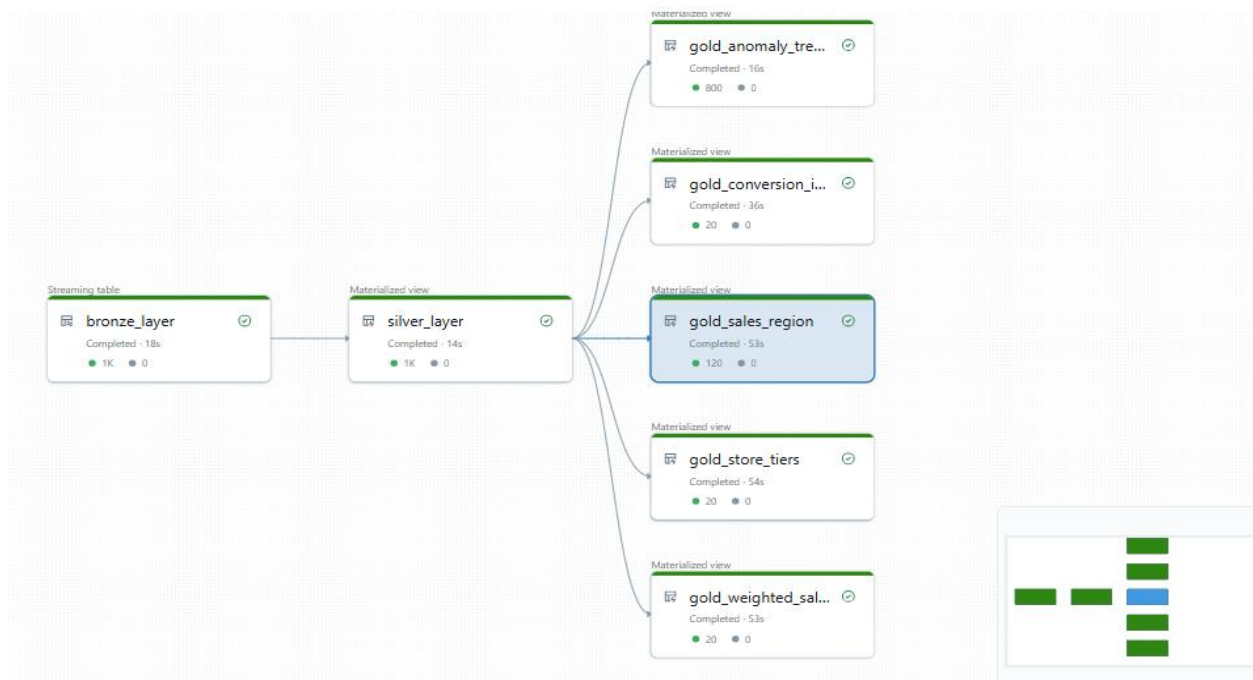
4. Store Tiering

Classifies stores into High, Medium, or Low performers based on total sales.

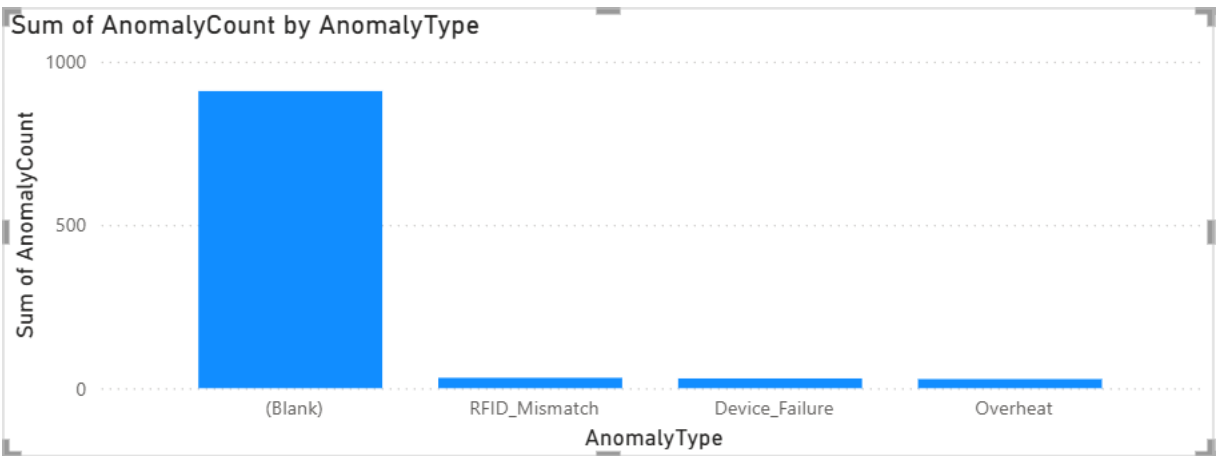
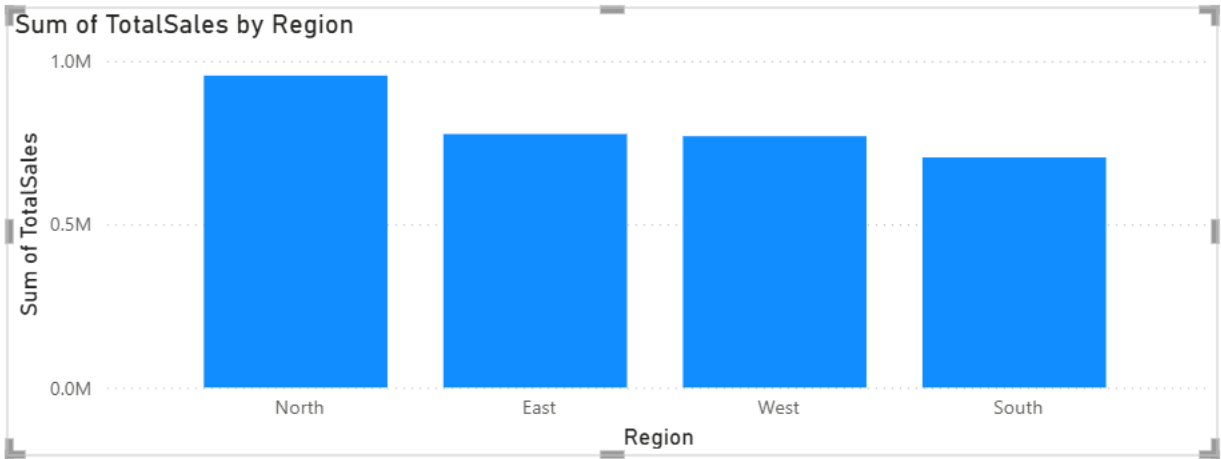
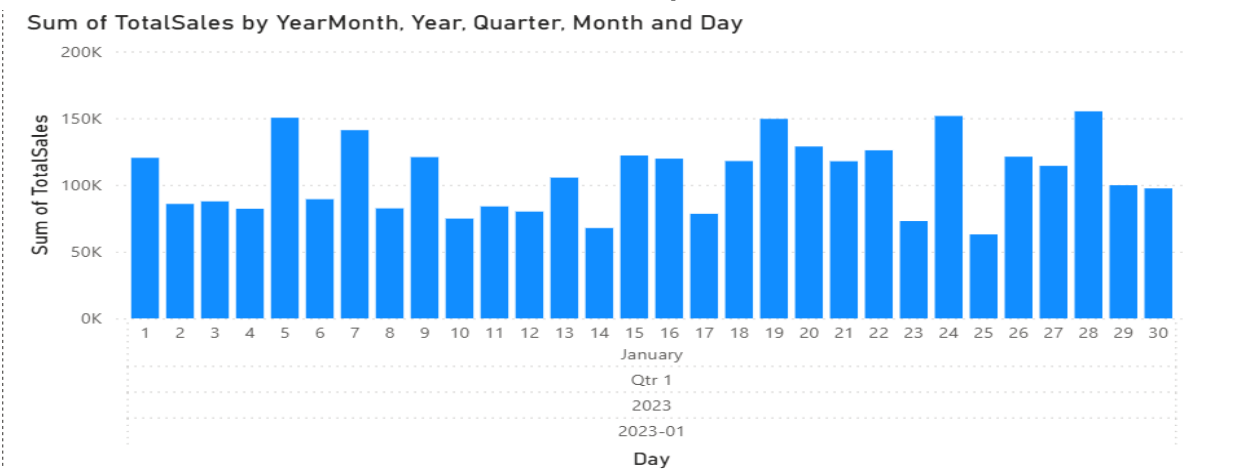
5. Sales vs Anomalies

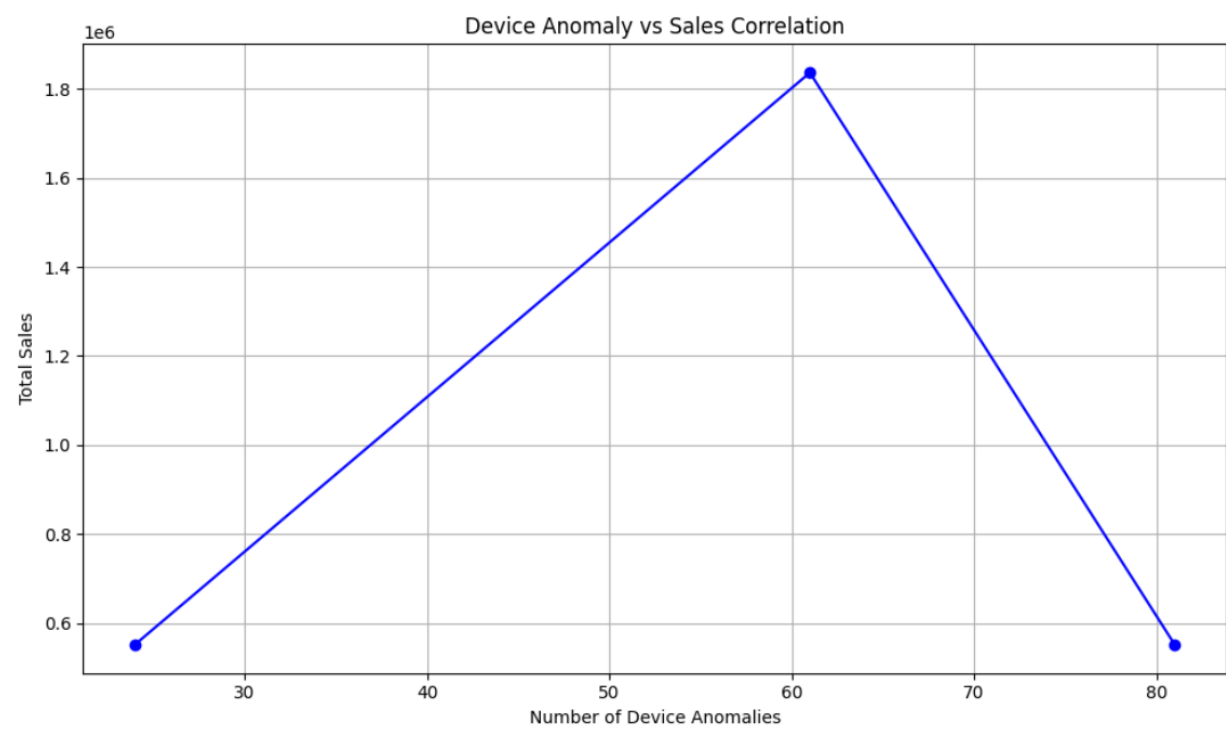
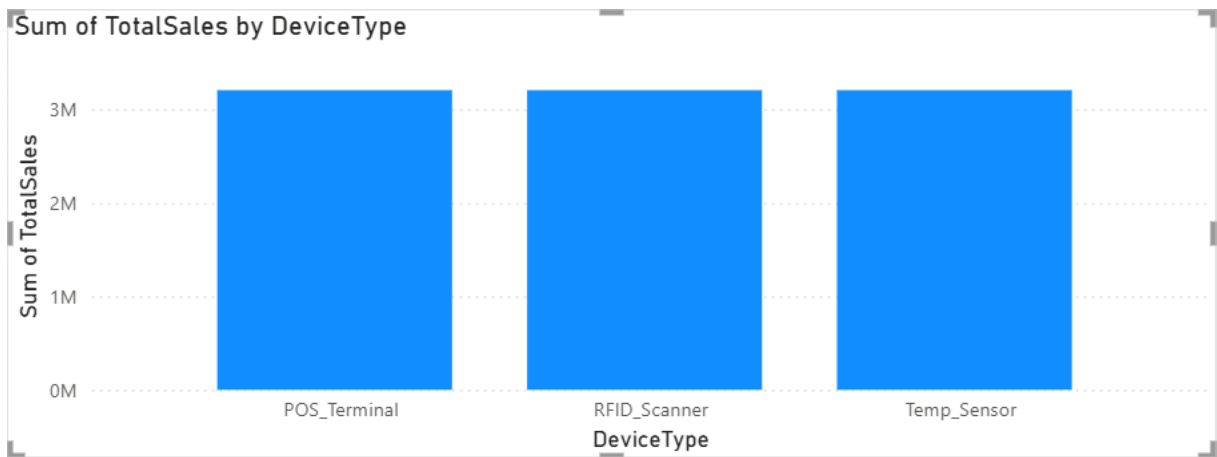
Computes the weighted average of sales per anomaly to assess operational efficiency.

Final Pipeline Structure:



Power BI Outputs:





Creation of Event Namespace and Eventhub

[Home](#) > [Event Hubs](#) >



Create Namespace

Event Hubs

Project Details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription *

LabsKraft



Resource group *

LabsKraft2027

[Create new](#)

Instance Details

Enter required settings for this namespace, including a price tier and configuring the number of units (capacity).

Namespace name *

sanyam-eventhub



.servicebus.windows.net

Region *

West US 2



The region selected supports Availability zones. Your namespace will have Availability Zones enabled. [Learn more.](#)

Pricing tier *

Basic



[Browse the available plans and their features](#)

CODE 1:

```
# Event Hub connection parameters
```

```
eh_namespace = "sanyam-eventhub"
```

```
eh_name = "sanyam-event"
```

```
eh_sas_key_name = "RootManageSharedAccessKey"
```

```
eh_sas_key = "OVSFVLx92G+QMXHI+Ji7ww3H5+BROT0nu+AEhLJ9Tno="
```

```
connection_string =
```

```
f"Endpoint=sb://{eh_namespace}.servicebus.windows.net/;SharedAccessKeyName={eh_sas_key_name};SharedAccessKey={eh_sas_key};EntityPath={eh_name}"
```

CODE 2:

```
from pyspark.sql.types import StructType, StructField, StringType, DoubleType, TimestampType, IntegerType

from pyspark.sql import functions as F

# Define schema for Event Hub messages

event_schema = StructType([

    StructField("OrderID", StringType(), True),

    StructField("CustomerID", StringType(), True),

    StructField("Region", StringType(), True),

    StructField("StoreID", StringType(), True),

    StructField("Product", StringType(), True),

    StructField("Quantity", IntegerType(), True),

    StructField("UnitPrice", DoubleType(), True),

    StructField("SalesAmount", DoubleType(), True),

    StructField("OrderTime", StringType(), True),    # can cast to Timestamp later in Silver

    StructField("DeviceType", StringType(), True),

    StructField("Temperature", DoubleType(), True),

    StructField("DeviceStatus", StringType(), True),

    StructField("AnomalyFlag", IntegerType(), True),

    StructField("AnomalyType", StringType(), True)

])

# Read streaming data from Event Hub

stream_df = (spark.readStream

    .format("eventhubs")

    .option("eventhubs.connectionString", connection_string)

    .load()

)

# Event Hub messages are in 'body' column as binary, decode them

decoded_df = (stream_df

    .withColumn("body_str", F.col("body").cast("string"))

    .select(F.from_json("body_str", event_schema).alias("data"))

    .select("data.*")
```

```

)

# Write stream to Bronze Delta table

bronze_stream = (decoded_df.writeStream

    .format("delta")

    .outputMode("append")

    .option("checkpointLocation", "/mnt/delta/events_bronze/_checkpoints/")

    .table("sanyam_capstone1_catalog.capstone_retail_sanyam.events_bronze") # new bronze table for streaming

)

```

OUTPUT:

```

❗ [UNSUPPORTED_STREAMING_SOURCE_PERMISSION_ENFORCED] Data source eventhubs is not supported as a streaming source on
a shared cluster. SQLSTATE: 0A000
File <command-6530246769983528>, line 41
30 decoded_df = (stream_df
31     .withColumn("body_str", F.col("body").cast("string"))
32     .select(F.from_json("body_str", event_schema).alias("data"))
33     .select("data.*")
34 )
35
36 # Write stream to Bronze Delta table
37 bronze_stream = (decoded_df.writeStream
38     .format("delta")
39     .outputMode("append")
40     .option("checkpointLocation", "/mnt/delta/events_bronze/_checkpoints/")
---> 41     .table("sanyam_capstone1_catalog.capstone_retail_sanyam.events_bronze") # new bronze table for streaming
42 )

```

TERMINAL:

Nano generate_data.py

CODE:

```

import random
import time
import uuid
from datetime import datetime
from azure.eventhub import EventHubProducerClient, EventData

# -----
# CONFIGURATION
# -----
EVENT_HUB_CONNECTION_STR =
"Endpoint=sb://hetulproject01eventspace.servicebus.windows.net;/SharedAccessKeyName=hello;SharedAccessKey=Cqw6MOblR0mll
hVGqOjbbEtH6s6lvgl2+AEhDNW7r4=;EntityPath=sanyam-event"
EVENT_HUB_NAME = "sanyam-event"

```

```

# -----
# Function to generate one IoT row
# -----
def generate_row():
    quantity = random.randint(1, 10)
    unit_price = round(random.uniform(50, 500), 2)
    sales_amount = round(quantity * unit_price, 2)
    device_status = random.choice(["Active", "Inactive", "Error"])
    anomaly_flag = 1 if device_status != "Active" else 0
    anomaly_type = random.choice(["Overheat", "LowBattery", "Disconnected"]) if anomaly_flag else "None"

    return {
        "OrderID": str(uuid.uuid4()),
        "CustomerID": f"CUST{random.randint(1000,9999)}",
        "Region": random.choice(["North","South","East","West"]),
        "StoreID": random.choice(["Store001","Store002","Store003"]),
        "Product": random.choice(["Laptop","Phone","Monitor","Keyboard","Mouse"]),
        "Quantity": quantity,
        "UnitPrice": unit_price,
        "SalesAmount": sales_amount,
        "OrderTime": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        "DeviceType": random.choice(["SensorA","SensorB","SensorC"]),
        "Temperature": round(random.uniform(20, 100), 2),
        "DeviceStatus": device_status,
        "AnomalyFlag": anomaly_flag,
        "AnomalyType": anomaly_type
    }

# -----
# Event Hub Producer
# -----
producer = EventHubProducerClient.from_connection_string(
    conn_str=EVENT_HUB_CONNECTION_STR,
    eventhub_name=EVENT_HUB_NAME
)

# -----
# Send data every second
# -----
try:
    for _ in range(1000): # Send exactly 1000 rows
        row = generate_row()
        event_data = EventData(str(row)) # convert dict to string
        with producer:
            producer.send_batch([event_data])
        print(f"Sent row: {row}")
        time.sleep(1) # 1-second interval
except KeyboardInterrupt:
    print("Streaming stopped by user.")
finally:
    producer.close()

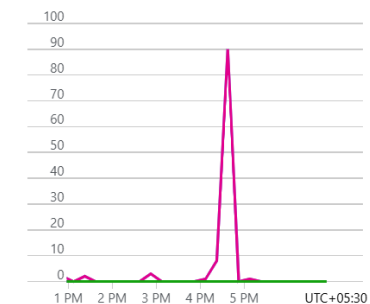
```


GENERATION AND LOAD of Random Data every second:

```
Sent row: {'OrderID': 'be2dac22-74a4-49b8-b2db-85422fe33f25', 'CustomerID': 'CUST2391', 'Region': 'South', 'StoreID': 'Store001', 'Product': 'Keyboard', 'Quantity': 8, 'UnitPrice': 287.59, 'SalesAmount': 2300.72, 'OrderTime': '2025-08-25 11:11:51', 'DeviceType': 'SensorA', 'Temperature': 86.54, 'DeviceStatus': 'Error', 'AnomalyFlag': 1, 'AnomalyType': 'Disconnected'}
Sent row: {'OrderID': 'bd87247e-b6cd-4541-a077-c6cb76614536', 'CustomerID': 'CUST5300', 'Region': 'North', 'StoreID': 'Store002', 'Product': 'Monitor', 'Quantity': 10, 'UnitPrice': 52.83, 'SalesAmount': 528.3, 'OrderTime': '2025-08-25 11:11:54', 'DeviceType': 'SensorC', 'Temperature': 34.27, 'DeviceStatus': 'Inactive', 'AnomalyFlag': 1, 'AnomalyType': 'LowBattery'}
Sent row: {'OrderID': 'e791c6d7-5e15-4a73-9618-59e8ad913fa7', 'CustomerID': 'CUST2701', 'Region': 'North', 'StoreID': 'Store001', 'Product': 'Phone', 'Quantity': 3, 'UnitPrice': 348.2, 'SalesAmount': 1044.6, 'OrderTime': '2025-08-25 11:11:57', 'DeviceType': 'SensorB', 'Temperature': 70.87, 'DeviceStatus': 'Inactive', 'AnomalyFlag': 1, 'AnomalyType': 'Overheat'}
Sent row: {'OrderID': 'f91bad2f-fb2d-4497-a2e5-99d029c6b9f3', 'CustomerID': 'CUST2645', 'Region': 'North', 'StoreID': 'Store002', 'Product': 'Laptop', 'Quantity': 10, 'UnitPrice': 341.51, 'SalesAmount': 3415.1, 'OrderTime': '2025-08-25 11:12:00', 'DeviceType': 'SensorC', 'Temperature': 68.17, 'DeviceStatus': 'Active', 'AnomalyFlag': 0, 'AnomalyType': 'None'}
Sent row: {'OrderID': 'ba5bed8a-fc54-427b-98de-da709875cbe7', 'CustomerID': 'CUST5846', 'Region': 'West', 'StoreID': 'Store001', 'Product': 'Monitor', 'Quantity': 1, 'UnitPrice': 76.26, 'SalesAmount': 76.26, 'OrderTime': '2025-08-25 11:12:03', 'DeviceType': 'SensorB', 'Temperature': 65.53, 'DeviceStatus': 'Error', 'AnomalyFlag': 1, 'AnomalyType': 'LowBattery'}
Sent row: {'OrderID': '9d5bfc6d-c6c9-49fc-b1f1-1b0ff6b08a20', 'CustomerID': 'CUST8727', 'Region': 'South', 'StoreID': 'Store003', 'Product': 'Mouse', 'Quantity': 1, 'UnitPrice': 417.45, 'SalesAmount': 417.45, 'OrderTime': '2025-08-25 11:12:06', 'DeviceType': 'SensorB', 'Temperature': 74.09, 'DeviceStatus': 'Inactive', 'AnomalyFlag': 1, 'AnomalyType': 'LowBattery'}
```

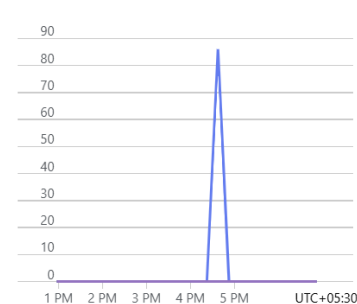
Show data for the last: 1 hour 6 hours 12 hours 1 day 7 days 30 days

Requests



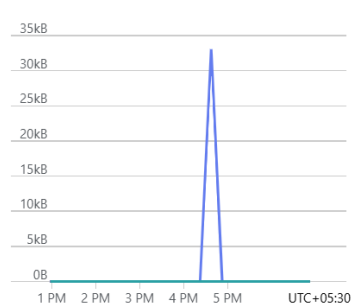
1/2
^ Incoming Requests (Sum), sanyam-ev... 105
v Successful Requests (Sum), sanyam-e... 105
■ Server Errors (Sum), sanyam-eventhub 0

Messages



1/2
^ Incoming Messages (Sum), sanyam-e... 86
v Outgoing Messages (Sum), sanyam-e... 0
■ Captured Messages (Sum), sanyam-... 0

Throughput



Incoming Bytes. (Sum), sanyam-ev... 33.07kB
Outgoing Bytes. (Sum), sanyam-even... 0B
Captured Bytes. (Sum), sanyam-event... 0B

Summary: This code sets up a real-time data pipeline from Azure Event Hub to Databricks. It reads streaming IoT retail data, decodes the binary messages into structured JSON using a defined schema, and extracts relevant fields. The processed data is then written to a Bronze Delta table in append mode. This enables continuous ingestion of live events for downstream analytics in the medallion architecture.